TP1 — Configuration avancée du système Linux

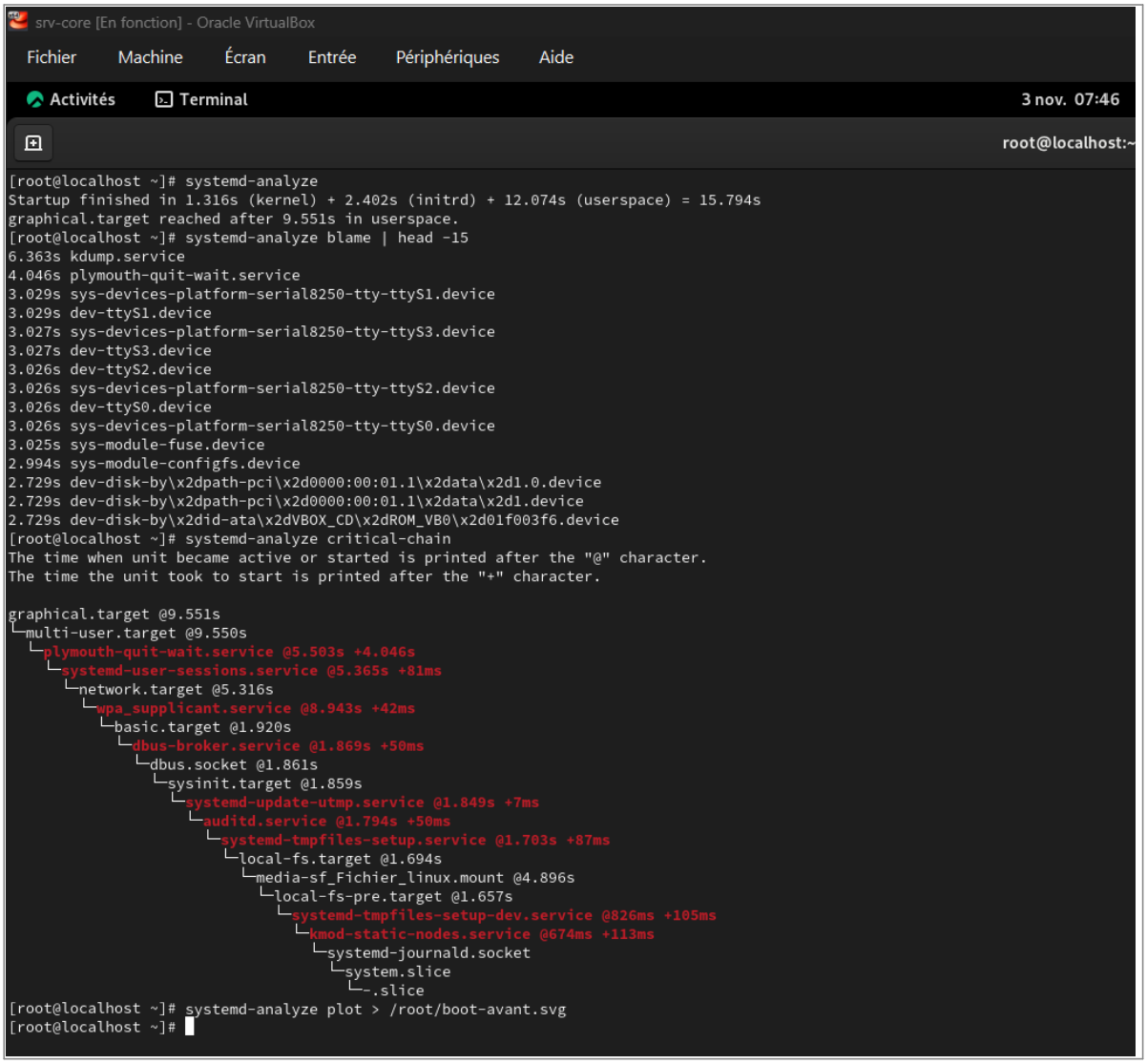
Module : Administration d'un Linux Server · BTC2 — Jour 1/5
Auteur : [Nom Prénom]
Date : 3 novembre 2026
Environnement : `srv-core` (Rocky Linux 9) et `srv-edge` (Ubuntu) sous VirtualBox

Mission 1 — Services, processus et séquence de démarrage

1.1 Chiffrer le démarrage, puis l'alléger

Je mesure d'abord le démarrage de `srv-core` avec `systemd-analyze`, `blame` et `critical-chain`, puis je génère le diagramme `boot-avant.svg`.

Capture : Démarrage avant optimisation



Le démarrage prend **15,794 s** au total (kernel 1,316 s + initrd 2,402 s + userspace 12,074 s), et `graphical.target` est atteint à **9,551 s**. Le `blame` place `kdump.service` en tête (6,363 s), suivi de `plymouth-quit-wait.service` (4,046 s). La chaîne critique montre aussi `media-sf_Fichier_linux.mount` (`@4,896 s`), qui est le montage du dossier partagé VirtualBox. Les nombreuses unités `*.device` (environ 3 s) correspondent à la détection du matériel : elles démarrent en parallèle et ne représentent pas un coût réel.

J'applique ensuite deux changements, puis je redémarre :

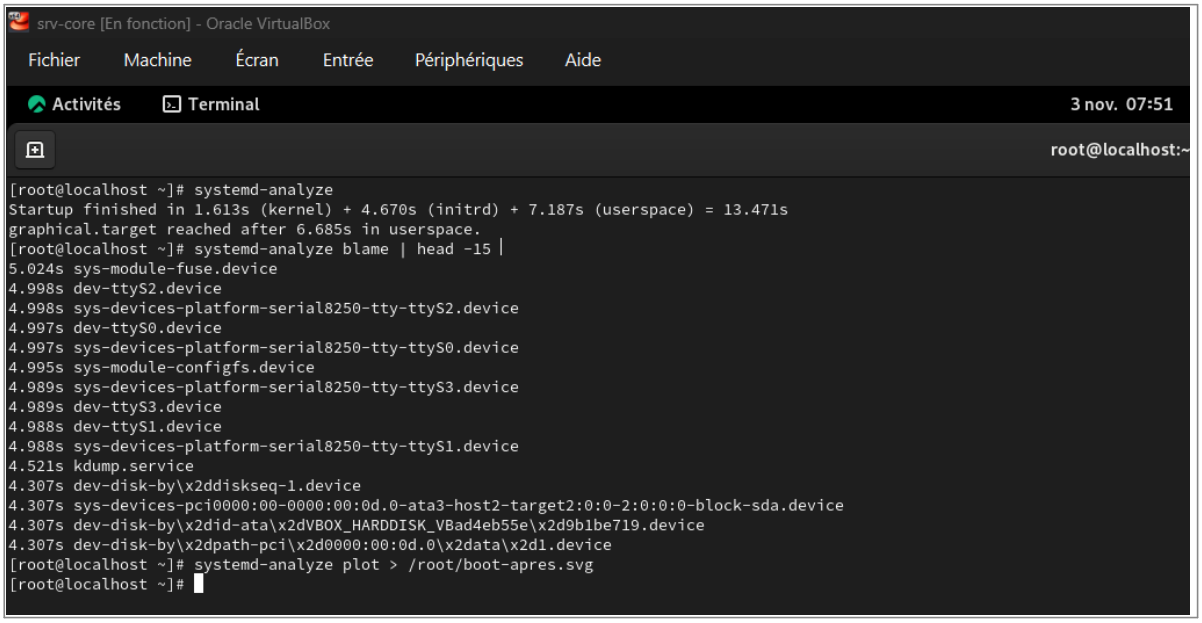
```
systemctl disable --now firewalld
```

```
systemctl mask NetworkManager-wait-online.service
systemctl reboot
```

- `firewalld.service` : Action : `disable --now` ; Justification : Désactivé **temporairement** pour ne pas filtrer les tests réseau de la Mission 2 (bond, VLAN, ping, `tcpdump`). Cela expose le serveur : il sera **réactivé au TP3**.

- `NetworkManager-wait-online.service` : Action : `mask` ; Justification : Il bloque `network-online.target` tant qu'une interface n'est pas configurée, ce qui retarde le démarrage. Le collecteur n'a pas besoin du réseau (il n'écrit que dans `/var/lib/helios`), donc l'attente est inutile. Le `mask` empêche aussi sa réactivation par dépendance.

Capture : Démarrage après optimisation



Le démarrage prend **13,471 s** (kernel 1,613 s + initrd 4,670 s + userspace 7,187 s), et `graphical.target` est atteint à **6,685 s**. `kdump.service` passe à 4,521 s dans le `blame`.

- **Kernel** : Avant : 1,316 s ; Après : 1,613 s ; Écart : +0,297 s
- **initrd** : Avant : 2,402 s ; Après : 4,670 s ; Écart : +2,268 s
- **Userspace** : Avant : 12,074 s ; Après : 7,187 s ; Écart : **-4,887 s**
- **Total** : Avant : **15,794 s** ; Après : **13,471 s** ; Écart : **-2,323 s (-14,7 %)**
- `graphical.target` : Avant : 9,551 s ; Après : 6,685 s ; Écart : **-2,866 s (-30 %)**

Analyse. Mes modifications portent sur l'espace utilisateur : la mesure pertinente est donc `userspace` (-4,9 s) et le temps d'atteinte de `graphical.target` (-2,9 s). Les phases kernel et initrd ne sont pas concernées : leur variation vient de la machine virtuelle (cache disque, charge de l'hôte).

Limite de la mesure. Je n'ai mesuré qu'un démarrage avant et un après. Ni `firewalld` ni `NetworkManager-wait-online` n'apparaissent dans les 15 premières lignes du `blame` « avant » : une partie du gain peut donc venir de la variabilité entre deux démarrages. Pour confirmer, il faudrait répéter la mesure plusieurs fois.

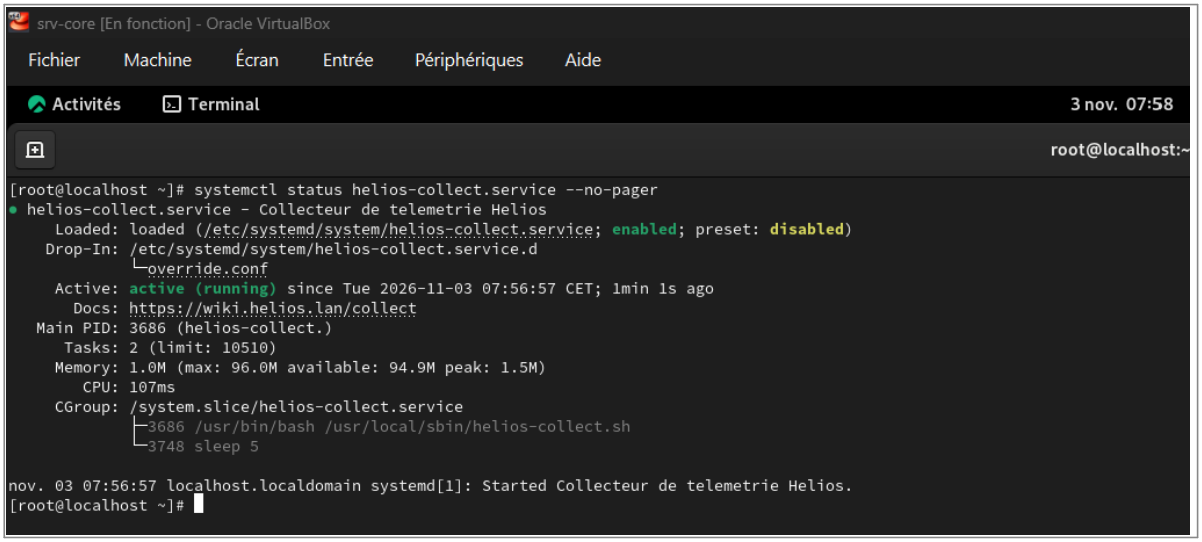
Unités pénalisantes identifiées : `kdump.service` (6,4 s, peu utile sur une VM de TP), `plymouth-quit-wait.service` (4,0 s, écran de démarrage graphique) et `media-sf_Fichier_linux.mount` (montage du dossier partagé, sur la chaîne critique).

Les diagrammes complets sont fournis dans `captures/boot-avant.svg` et `captures/boot-apres.svg`.

1.2 Créer le service de collecte

Je crée un compte système dédié `helios` (`--system`, shell `/sbin/nologin`) pour que le collecteur ne tourne plus en root, puis j'active le service. L'unit `helios-collect.service` utilise `Restart=on-failure`, `RestartSec=5s` et plusieurs directives de durcissement : `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=strict`, `ProtectHome`, `ProtectKernelTunables`, `ProtectControlGroups`, `RestrictSUIDSGID`, et `CapabilityBoundingSet=` vide.

Capture : État du service après activation



Le service est `loaded ... enabled`, donc il démarrera au boot. Il est `active (running)`, le drop-in `override.conf` est chargé (ligne `Drop-In`), et il consomme 1,0 Mo pour une limite de 96,0 Mo. Son cgroup `/system.slice/helios-collect.service` contient 2 processus : le script et son `sleep 5`.

Score de sécurité. Pour avoir un point de comparaison, j'ai mesuré une unit minimale temporaire sans aucune protection (`helios-nu.service`), puis le vrai service.

Capture : Score de l'unit non durcie (avant)



Score de **9,6 — UNSAFE** : l'unit tourne en root sans aucune restriction.

Capture : Score de l'unit durcie (après)



Score de **5,3 — MEDIUM** : le sandboxing et l'utilisateur non privilégié font baisser l'exposition de 4,3 points.

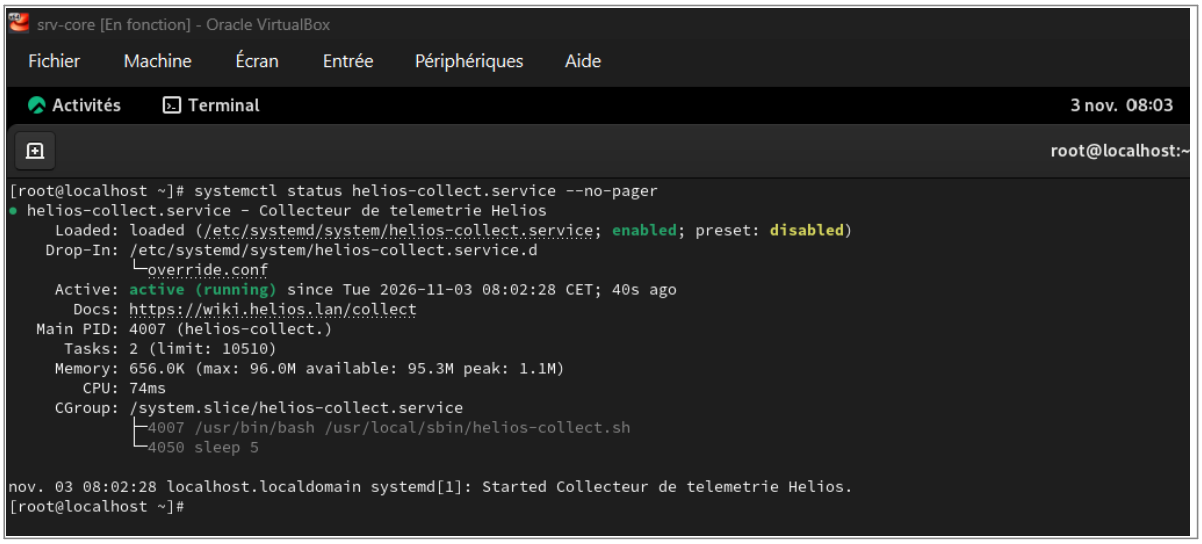
- `helios-nu.service` **(avant)** : Score : 9,6 ; Niveau : UNSAFE
- `helios-collect.service` **(après)** : Score : 5,3 ; Niveau : MEDIUM

Le score reste à 5,3 : d'autres directives (restriction des familles d'adresses réseau, filtrage des appels système, `PrivateDevices`, etc.) pourraient le réduire encore.

1.3 Redémarrage automatique et drop-in

Je tue le processus principal avec `kill -9`, puis j'attends plus longtemps que `RestartSec` (5 s).

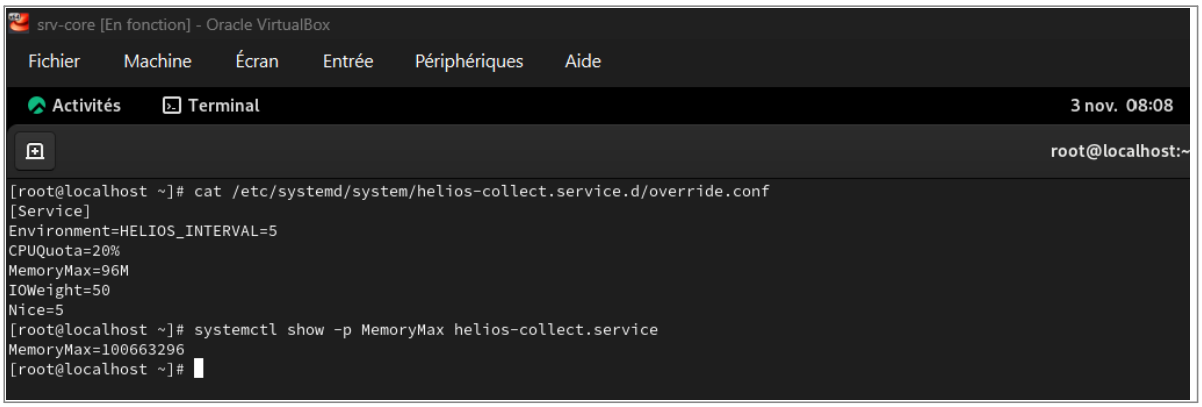
Capture : Service redémarré après `kill -9`



Après la destruction du processus, systemd a relancé le service tout seul : il est de nouveau `active (running)`, avec un nouveau PID (4007 contre 3686 auparavant) et une date de démarrage récente. C'est l'effet de `Restart=on-failure`.

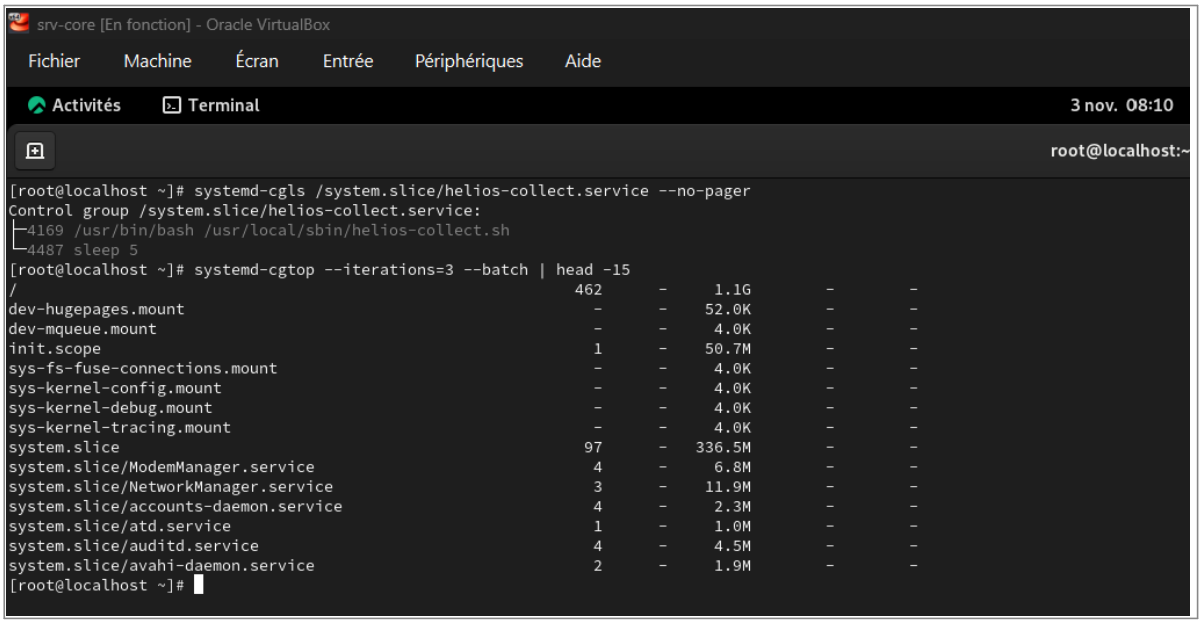
Les réglages locaux sont placés dans un **drop-in** et non dans l'unit d'origine, qui reste inchangée.

Capture : Drop-in et limite mémoire



Le contenu de `override.conf` montre les cinq directives ajoutées : intervalle de collecte de 5 s, `CPUQuota=20%`, `MemoryMax=96M`, `IOWeight=50` et `Nice=5`. La commande `systemctl show -p MemoryMax` renvoie **100663296**, soit 96 × 1024 × 1024 octets = 96 Mo : le drop-in est bien pris en compte, ce qui est le critère de réussite demandé.

Capture : Cgroup du service et consommation



`systemd-cgls` confirme que le service s'exécute dans son propre cgroup `/system.slice/helios-collect.service` avec 2 processus (le script `helios-collect.sh` et `sleep 5`). `systemd-cgtop` liste la consommation par cgroup. Les limites du drop-in s'appliquent donc à ce cgroup, et ne sont pas seulement écrites dans un fichier.

Capture : Score de sécurité après le drop-in

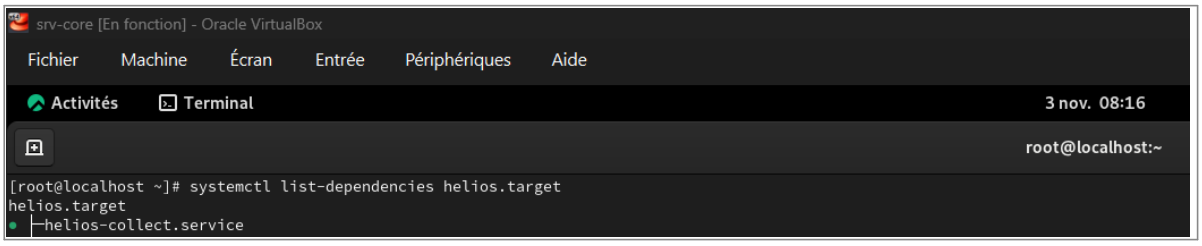


Le score est toujours de **5,3 — MEDIUM** : le drop-in n'ajoute que des limites de ressources (CPU, mémoire, E/S), que l'analyse de sécurité n'évalue pas.

1.4 Cible métier et priorités de processus

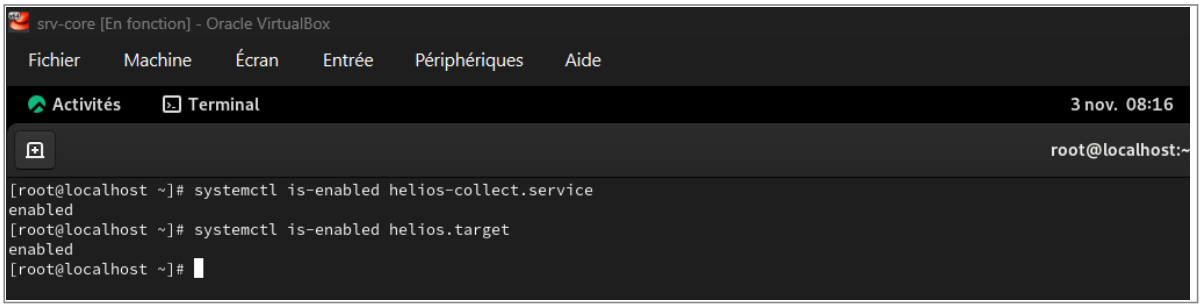
Je crée `helios.target`, j'y rattache le collecteur avec `systemctl add-wants`, puis j'active la target.

Capture : Dépendances de la target



`helios-collect.service` apparaît bien sous `helios.target`, avec un point vert : il est actif et rattaché à la pile applicative.

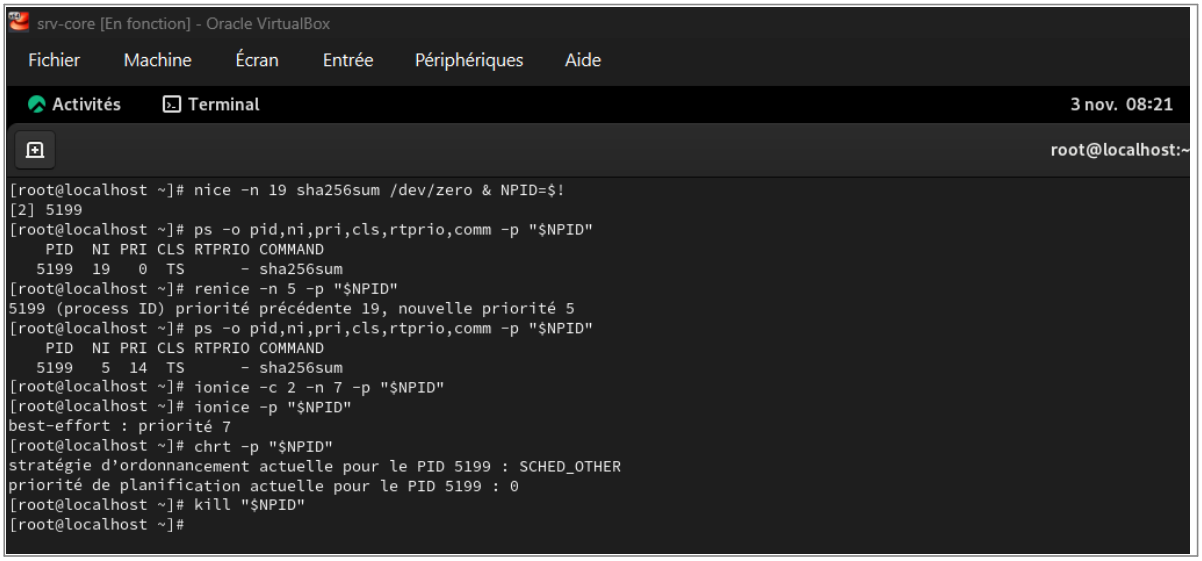
Capture : Activation au démarrage



Le service et la target renvoient `enabled` : ils démarreront automatiquement au boot.

Je modifie ensuite à chaud la priorité d'un processus gourmand (`sha256sum /dev/zero`) avec `nice`, `renice`, `ionice` et `chrt`.

Capture : Priorités CPU et E/S d'un processus



Au lancement (`nice -n 19`), le processus a `NI = 19` et `PRI = 0`. Après `renice -n 5`, il passe à `NI = 5` et `PRI = 14` : il est devenu plus prioritaire. `ionice -c 2 -n 7` puis `ionice -p` affichent la classe **best-effort, priorité 7**. `chrt -p` indique `SCHED_OTHER` avec une priorité de 0 : le processus n'est pas en temps réel. Le `kill` final l'arrête.

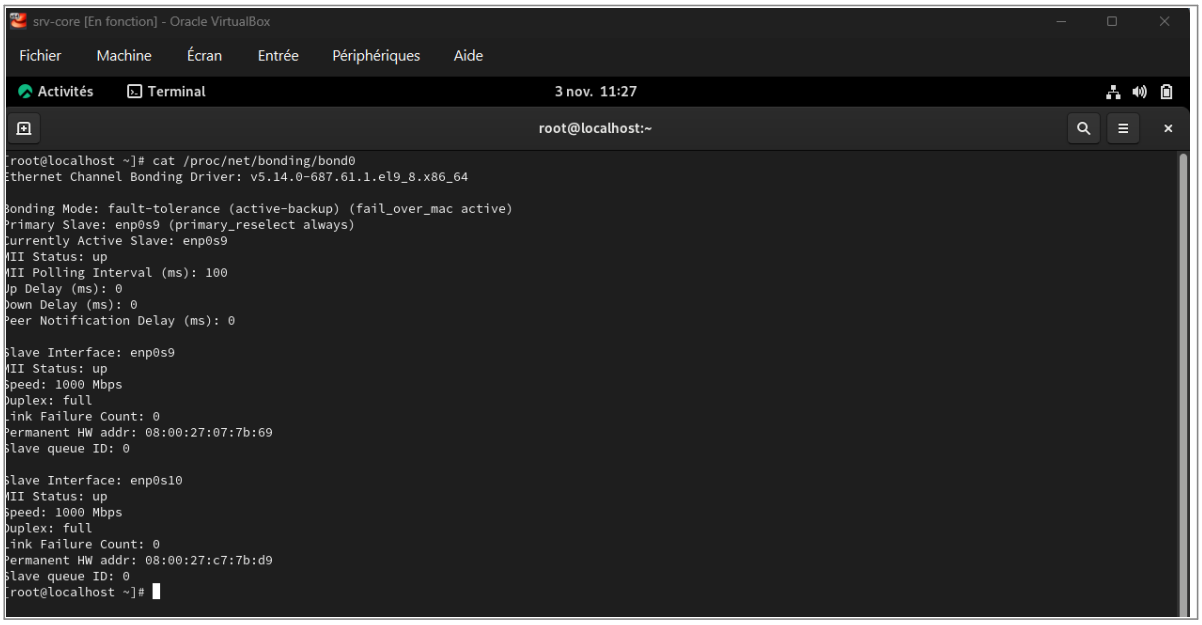
- `nice` / `renice` : priorité d'accès au **CPU** (-20 à 19).
- `ionice` : priorité d'accès au **disque**.
- `chrt` : **politique** d'ordonnancement (`SCHED_OTHER` standard, ou `SCHED_FIFO` / `SCHED_RR` en temps réel).

Mission 2 — Réseau avancé : bond, VLAN, routage par règles

2.1 Agrégation active-backup

Sur `srv-core`, je crée le bond `bond0` en mode `active-backup` avec deux ports : `enp0s9` (port primaire) et `enp0s10`. Le lien est surveillé toutes les 100 ms (`miimon=100`) et l'adresse du bond est `10.42.0.10/24`.

Capture : État du bond avant la coupure

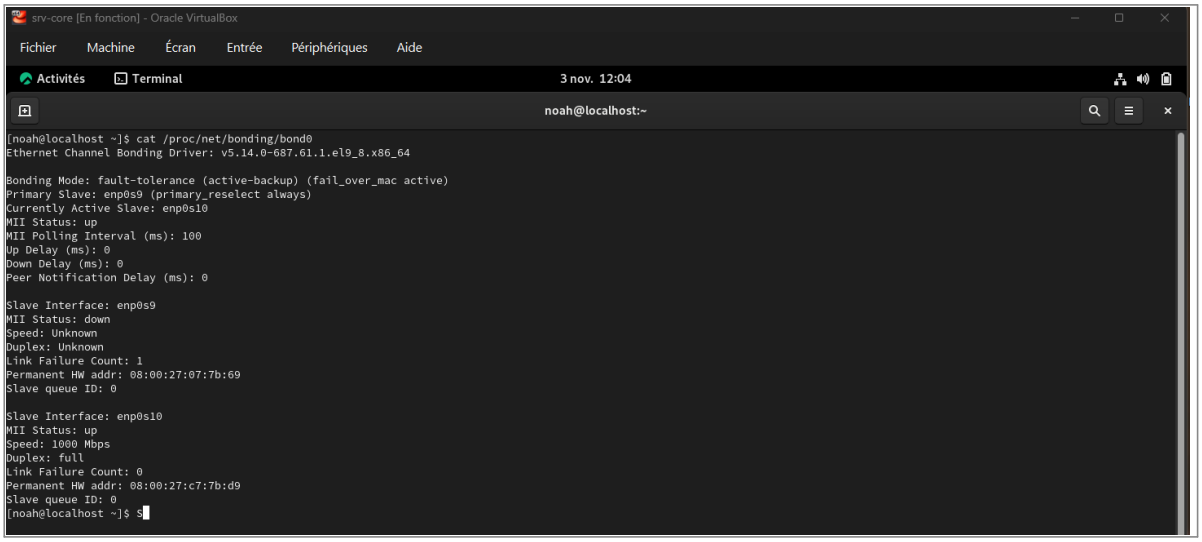


Le mode est bien `fault-tolerance (active-backup)`, le port primaire est `enp0s9` et c'est aussi le **port actif** (`Currently Active Slave: enp0s9`). Le bond est `MII Status: up`, avec un intervalle de surveillance de 100 ms. Les deux ports sont `up` (1000 Mbps, full duplex) avec `Link Failure Count: 0`, ce qui correspond au critère de réussite : deux ports up et un port actif.

2.2 Provoquer la bascule

Je coupe le câble virtuel de l'adaptateur 3 (`enp0s9`, le port actif) depuis l'hôte avec `VBoxManage controlvm srv-core setlinkstate3 off`, puis je relis l'état du bond.

Capture : État du bond pendant la coupure

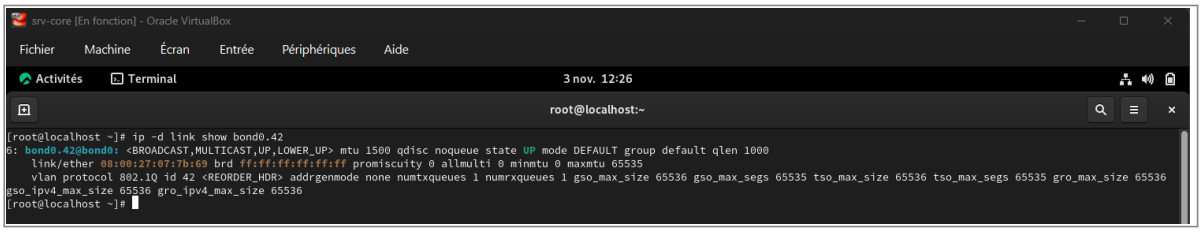


Le port `enp0s9` est passé en `MII Status: down` (`Speed: Unknown`) et son `Link Failure Count` vaut maintenant **1**. Le bond a basculé automatiquement : `Currently Active Slave` est devenu `enp0s10`, qui est `up`. Le bond reste lui-même en `MII Status: up` : le trafic continue de passer par le port de secours. Le primaire reste `enp0s9` avec `primary_reselect always` : il redevient actif dès que son lien est rétabli.

2.3 VLAN 802.1Q « transport » (id 42)

Je crée sur le bond la sous-interface `bond0.42` (VLAN 42) avec l'adresse `10.42.42.10/24`.

Capture : Sous-interface VLAN 42



La ligne `vlan protocol 802.1Q id 42` confirme que `bond0.42` est bien une interface VLAN d'identifiant 42. Elle est rattachée à `bond0` (`bond0.42@bond0`) et son état est `UP`.

2.4 Table de routage secondaire et règle de sélection

Je déclare la table `142 transport`, j'y place une route par défaut via `srv-edge` (`10.42.42.11`) et j'ajoute une règle : ce qui part de l'adresse `10.42.42.10` consulte cette table.

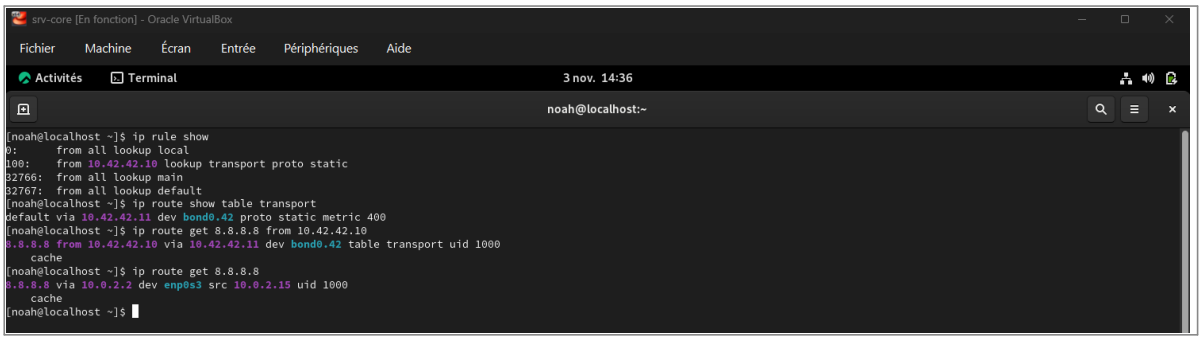
Capture : Règle, table transport et sélection de route



`ip rule show` contient la règle `100: from 10.42.42.10 lookup transport`. La table `transport` contient `default via 10.42.42.11 dev bond0.42`. Le test `ip route get 8.8.8.8 from 10.42.42.10` sort par `bond0.42` via `10.42.42.11` (table transport). Le même test sans `from` sort par `enp0s3` via `10.0.2.2` (route par défaut NAT). C'est donc bien l'adresse source qui décide du chemin.

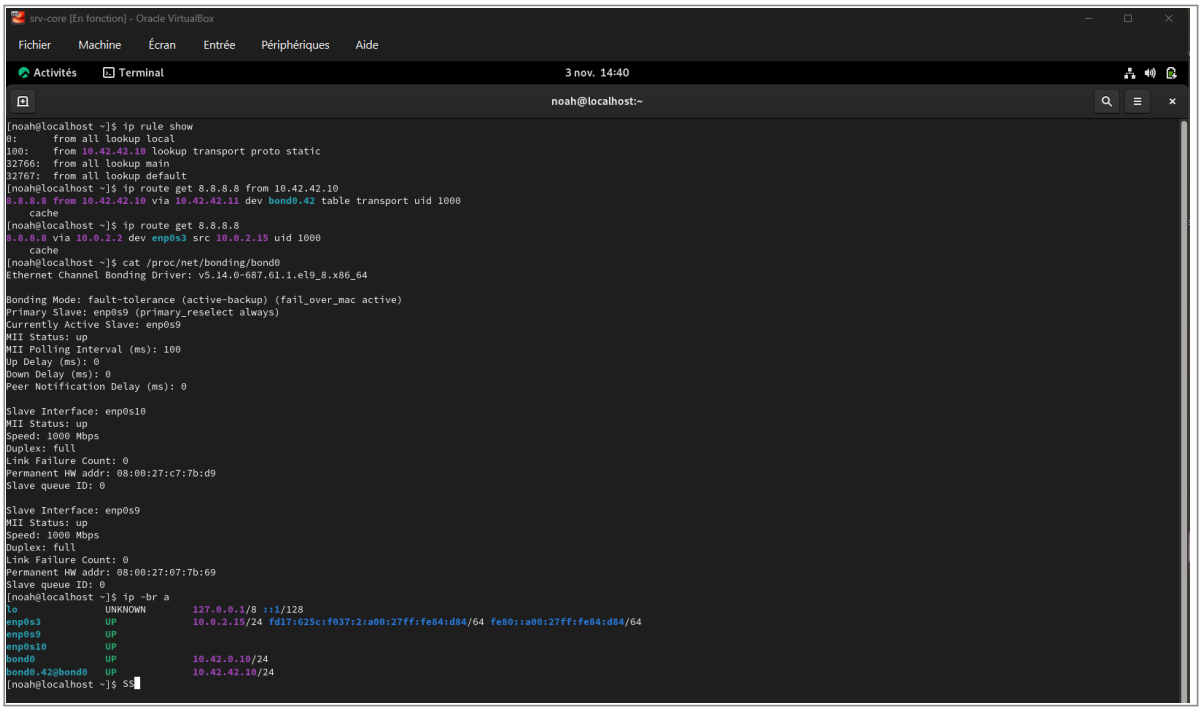
Persistance. Je redémarre `srv-core` puis je refais les mêmes vérifications.

Capture : Routage après redémarrage



Après le redémarrage, la règle `100`, la table `transport` et les deux `ip route get` donnent exactement les mêmes résultats : la configuration de routage est persistante.

Capture : Bond et adresses après redémarrage



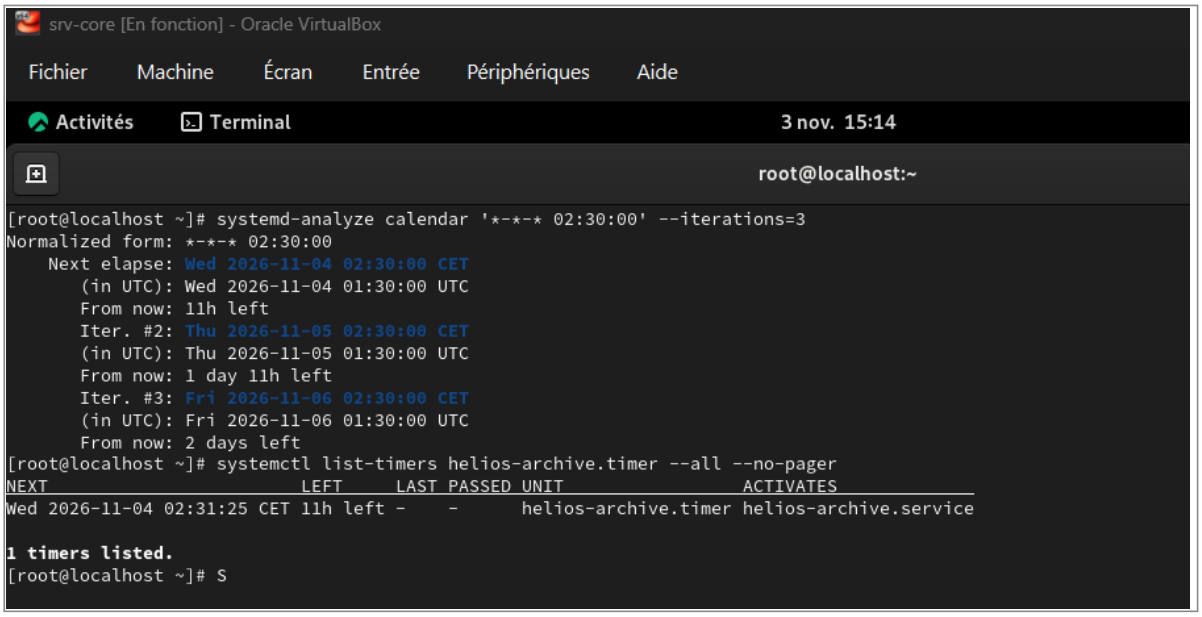
Après le redémarrage, `bond0` est de nouveau actif avec `enp0s9` comme port actif et les deux ports `up`. `ip -br a` montre `bond0` en `10.42.0.10/24` et `bond0.42` en `10.42.42.10/24` (les deux ports du bond n'ont pas d'adresse propre, ce qui est normal). L'ensemble de la configuration réseau survit au redémarrage.

Mission 3 — Automatisation : script robuste et timer systemd

3.2 Planification par timer systemd

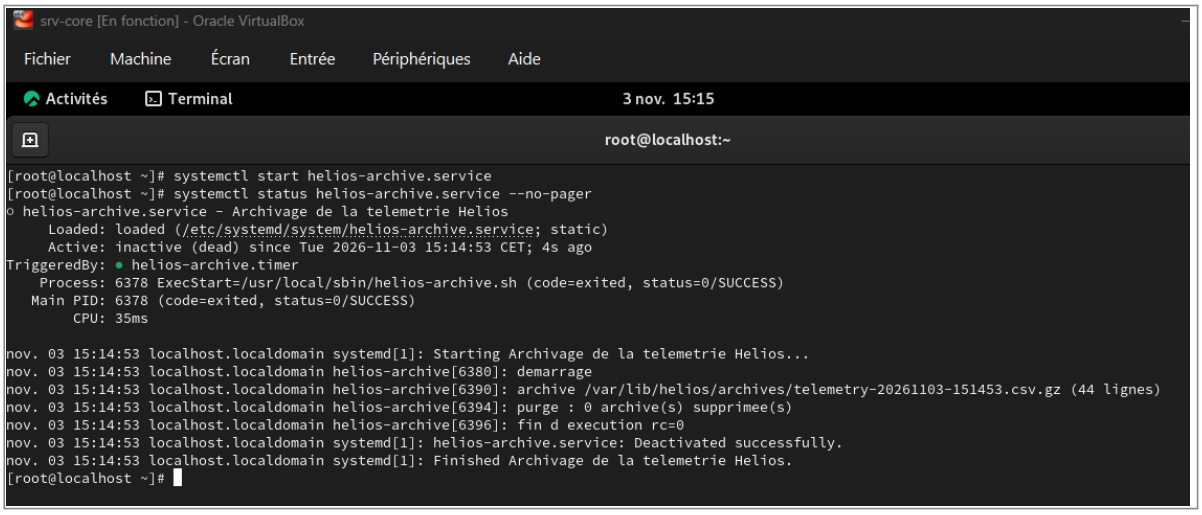
J'active `helios-archive.timer` : déclenchement tous les jours à 02:30 (`OnCalendar=-*-* 02:30:00`), avec un décalage aléatoire (`RandomizedDelaySec=120`) et un rattrapage en cas d'extinction (`Persistent=yes`).

Capture : Planification du timer



`systemd-analyze calendar` donne la forme normalisée `--* 02:30:00` et les trois prochaines exécutions, chaque jour à 02:30 (4, 5 et 6 novembre). `systemctl list-timers` affiche `helios-archive.timer` avec une prochaine exécution (**NEXT**) le mercredi 4 novembre à **02:31:25**, dans 11 h, et il active `helios-archive.service`. Les 85 secondes après 02:30 viennent du `RandomizedDelaySec` (120 s au maximum) : le NEXT est bien cohérent avec 02:30.

Capture : Exécution du service d'archivage



Je lance le service à la main avec `systemctl start`. Il apparaît `inactive (dead)` : c'est normal, car c'est un service `Type=oneshot` qui s'arrête une fois le travail terminé. Le script s'est bien terminé (`code=exited, status=0/SUCCESS`) et le service est rattaché au timer (`TriggeredBy: helios-archive.timer`). Le journal montre le déroulé : démarrage, archive créée `/var/lib/helios/archives/telemetry-20261103-151453.csv.gz` (44 lignes), purge de 0 archive, puis `fin d execution rc=0`.